

This will run all the requests in *.rcq files in the current working directory, and save the responses (*.rcs) in the /path/to/responseDir. The command line client will also print a summary of the test executions.

Extending RESTClient

During the development of version 2.3, the code was re-factored to a more modular form for extensibility. Now the code is organised into various modules managed by Maven. The *restclient-lib* module has the core functionality of RESTClient. Having this as the dependency, various interfaces have been developed (the RESTClient GUI, CLI and Ant plug-in being examples). To demonstrate the ease of the API, I will show you how to write the code to execute a request and write the response-body in the console. First add the dependency for *restclient-lib* in your Maven project [for a detailed discussion on setting up the environment, refer to the Cook Book: code.google.com/p/rest-client/wiki/Cookbook]:

```
<dependency>
<groupId>org.wiztools.restclient</groupId>
<artifactId>restclient-lib</artifactId>
<version>2.3</version>
</dependency>
```

Next, write the following code to execute the request:

```
import org.wiztools.restclient.Request;
import org.wiztools.restclient.RequestBean;
import org.wiztools.restclient.HTTPMethod;
import org.wiztools.restclient.View;
import org.wiztools.restclient.Implementation;
import org.wiztools.restclient.RequestExecutor;

// Step 1: Create the request:
RequestBean requestBean = new RequestBean();
requestBean.setUri(new java.net.URL("http://wiztools.org/"));
requestBean.setMethod(HTTPMethod.GET);

Request request = requestBean;

// Step 2: Write the handler
View view = new View();
@Override
public void doStart(Request request){
    // do nothing!
}

@Override
public void doResponse(Response response){
    System.out.println(response.getResponseBody());
}
```

```
@Override
public void doCancelled(){
    // do nothing!
}

@Override
public void doEnd(){
    // do nothing!
}

@Override
public void doError(final String error){
    System.err.println(error);
}
};
```

```
// Step 3: Execute:
RequestExecutor executor = Implementation.of(RequestExecutor.class);
executor.execute(request, view);
```

This example is taken from the *RESTClient Cook Book* [code.google.com/p/rest-client/wiki/Cookbook]. The Cook Book has more details on extending.

What next?

Well, there's Ant and Maven integration. Ant integration work has begun but what about any contributors for Maven?

The team

Various people have contributed to RESTClient through suggestions, ideas, testing efforts, documentation and code contribution. I will not be able to list all of them. But the major contributors, besides me, are:

- Ravi Subramaniam: He was a young and energetic lad. Tragedy struck when we lost him in an accident in 2008. He had contributed the initial persistence code.
- Jacky Chan: No, he is not the actor! Jacky is from China, and contributed various bug fixes and modularisation ideas. But his biggest contribution is in making a RESTClient plug-in for IntelliJ IDEA.
- Velrajan: He has contributed some bug fixes and re-wrote the persistence code using XOM.

Other people who have made substantial contributions are Balasubramani S D and Avi Flax. As I said before, there have been other significant contributors, and I am thankful to all of them. 

By: Subhash Chandran

Subhash is a software developer working in the innovation department of the Chennai-based software house, Sella Synergy India Ltd. He has contributed to various Open Source projects, and publishes his contributions on his site WizTools.org. He also maintains a technical blog at indWiz.com. He may be contacted at: subviz@gmail.com.